Lab 0
Introduction to Git Source Control

Learning Objectives

By the end of this lab, students will be able to:

- Explain what source control is and why it is important.
- Initialize a Git repository.
- Check the status of a repository.
- Stage files for commit.
- Create meaningful commits.
- Push commits to a remote GitHub repository.
- Verify that changes appear on GitHub.

Prerequisites

Students should have:

- Git installed
- A GitHub account
- Visual Studio Code (or another editor)
- Internet connection

What is Git?

Git is a distributed version control system that helps track and manage changes in code over time. It allows multiple developers to work on the same project, view previous versions, and restore changes when needed.

Features of Git

1. Version Control System: Git keeps track of every change you make to your project files. You can go back to previous versions is any requirement arises.

2. Repositories: A Git repository (or repo) is like a project's central hub where everything related to your work is stored and managed. There are two main types:

- **Local Repository:** This is a copy of the repository which is stored in your computer. You can work on your project and make changes here.
- **Remote Repository:** This is stored on a server, like GitHub, where you and others can share and work on a project.

3. Commits: Every time you make changes and save them in Git, these are called commits. The repository keeps track of all the commits you have made.

4. Branches: A repository allows you to create Branches and work on new features and fixes. For example, you can have a "main" branch for stable code and a "feature" branch for new features you're developing.

5. Merging: Once you are done with the branches you have made, you can merge those branches into main branch. This adds the changes you have made in the rest of the project.

6. Cloning: Cloning is a process of making a complete copy of Git repository. It's like copying the entire project from central location (like website) to your own computer.

Reasons to Choose Git

- **Distributed System:** Every developer has a complete local copy of the project, allowing for offline work and increased resilience.
- **Performance:** Git is fast and efficient, handling large projects with ease.
- **Branching and Merging:** Git's lightweight branching and easy merging facilitate parallel development and feature isolation.
- **Collaboration:** Tools like GitHub enhance team collaboration, code reviews, and project management.
- **Track Changes:** Git tracks changes and maintains a history, making it easy to revert to previous versions. (Geeks, 2026)

Key Terms

Term	Description
Repository (Repo)	A project tracked by Git
Commit	A snapshot of your project
Stage	Selecting changes to include in the next commit
Push	Uploading commits to GitHub
Remote	The online GitHub repository

Steps to Setup a Git

Lab Part 1 – Create a Repository

Step 1

Install Git: Download and install Git from the official Git website.

- Create a new folder named: Lab1_GitLab
- Open the folder in VS Code.

Configure Git: Set up your username and email.

Open the terminal in VS Code and type:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Step 2

Initializing Git:

```
git init
```

Questions

1. What message appears after running git init?
2. What hidden folder was created?

Lab Part 2 – Create Your First File

Create a file called README.md. The file should be created within the project folder Lab1_GitLab

```
Lab1_GitLab
     |_______README.md
```

After the README.md file is created, add the following in the README.md file:

```
# Lab 1: Introduction to Git Source Control @ <type student's
full name>
```

Why README.md?

A README.md file is the standard documentation file for a Git repository. The .md extension stands for **Markdown**, a simple formatting language used by GitHub to display headings, lists, links, code blocks, and other formatted text. When someone visits your repository on GitHub, the README.md is typically the first file they see, making it the ideal place to describe your project.

Lab Part 3 – Check Repository Status

In the Terminal, run:

```
git status
```

Observe

Notice:

- README.md is untracked.
- Git knows a file was created.

Questions

1. What color is the filename?
2. What does "Untracked" mean?

Lab Part 4 – Stage Changes

To add a file to the staging area, use the git add command followed by the name of the file or folder you want to stage. If you want to stage all new and modified files in the current directory, use:

```
git add .
```

or

```
git add *
```

For this lab, stage the README.md file by running:

```
git add README.md
```

This command tells Git to include the current version of README.md in the next commit.

We can check the status of our repository by using the `git status` command.

```
git status
```

Observe

The file should now be listed under:

```
Changes to be committed
```

Question

1. Why do you think Git requires a staging area?

Lab Part 5 – Commit Changes

After we stage, we need to record the current state of the files we have staged, allowing you to track your project's history and revert to previous versions if needed. Each commit has a unique identifier (called a **commit hash**) and includes:

- The changes that were made.
- The author's name and email.
- The date and time of the commit.
- A commit message describing the changes.

Think of a commit as saving a checkpoint in your project's history. You can always return to that checkpoint later if needed.

To commit to the changes, we need to include a message to the commit by using **`-m`** as:

```
git commit -m "Initial commit"
```

In this command:

- `git commit` creates a new commit.
- `-m` tells Git that the next text is the commit message.
- `"Initial commit"` describes the changes included in the commit.

Observe

Git creates a permanent snapshot after the commit command. After it, we can use the git log to display the commit history of a repository. It lets you see all the commits that have been made, along with information about each one:

```
git log
```

Questions

1. What is the commit message?
2. What is display after running `git log`?

Lab Part 6 – Connect to GitHub

Login into GitHub and create a new GitHub repository called

`ET712_JS_programming`

- Create the repository as **private.**
- Do **not** initialize it with a README and copy the repository URL.

After that, copy the URL of the remote repository. Then, return to your local repository and connect it to the remote repository by running the following command:

```
git remote add origin https://github.com/username/
ET712_JS_programming.git
```

To verify the connection:

```
git remote -v
```

Lab Part 7 – Push to GitHub

The command **git push** sends your commits from the local repository to the remote repository.

```
git push -u origin main
```

Here's what each part means:

- **`git push`** sends your commits from the local repository to the remote repository.
- **`-u`** is a short for **`--set-upstream`**. It links your local main branch to the remote main branch so that future pushes and pulls can be done with simply git push or git pull.
- **`origin`** is the default name of the remote repository.
- **`main`** is the branch you are pushing.

Observe

Visit GitHub repository and refresh the repository page.

You should now see your README file.

Lab Part 8 – Make Another Change

In the local repository, open the README.md file and add the following:

```
Today I learned the basics of Git.
```

Save the file and check the status by typing in the Terminal:

```
git status
```

Observe

Visit GitHub repository and refresh the repository page.

Question:

1. Do you see the changes in the remote repository?

Stage the Change

```
git add README.md
```

Commit the Change

```
git commit -m "Updated README with learning summary"
```

Push Again

```
git push
```

Observe

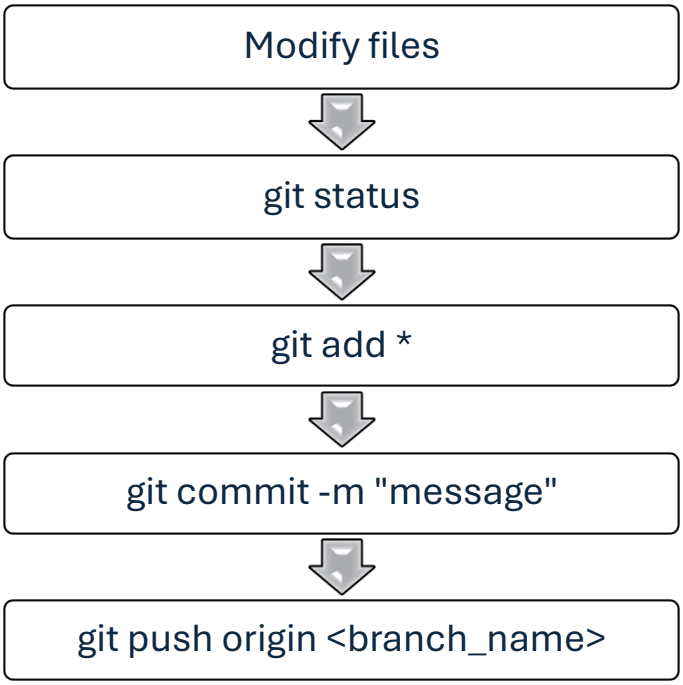
Visit GitHub repository and refresh the repository page.

Question:

2. Do you see the changes in the remote repository?

Git Workflow

The basic Git workflow is:



Common Git Commands

Command	Purpose
git init	Create a new Git repository
git status	Show repository status
git add <file>	Stage a file
git add .	Stage all changes
git commit -m "message"	Save a snapshot
git log	View commit history
git remote -v	View connected remotes
git push	Upload commits to GitHub
git pull	Download the latest changes
git clone <url>	Copy an existing repository

Reflection Questions

1. Why is source control important?
2. What is the purpose of a commit?
3. What is the difference between git add and git commit?
4. Why do developers use meaningful commit messages?
5. What happens when you run git push?
6. What would happen if you forgot to commit before pushing?

Submission Requirements

Students should submit:

- The URL of the GitHub repository.
- Answers of the following lab parts:
    - Lab Part 1 – Create a Repository (question 1 and 2)
    - Lab Part 2 – Create Your First File (question 1 and 2)
    - Lab Part 3 – Check Repository Status (question 1 and 2)
    - Lab Part 4 – Stage Changes (question 1)
    - Lab Part 5 – Commit Changes (question 1 and 2)
    - Lab Part 8 – Make Another Change (question 1 and 2)
- Answers to the reflection questions (question 1 through 6).

Grading Criteria

- The URL of the GitHub repository (including README.md): 3 pts
- Answers of lab parts: 4 pts
- Reflection questions: 3 pts